

Depression Storage Overhaul

Andy Bock and Cory Russell

Status

- 2/17 – Finished Level One, starting first two functions of level two

Goals :

1. Simplify, “efficiencify” and move depression storage calculation to open source
2. Test new datasets (DSWE) and ideas (see below)
3. Integrate NCMapper from Canada

Needed Inputs

- HRUS
- Stream Segments
- waterbodies
 - waterbodies
 - NHDPv2
 - NHDPHR
 - *DSWE (waiting on Anthoneh’s workflow)*
 - *Backup is Nate’s workflow, contact if we don’t have after working on functions below*
 - Other datasets (lagos, etc.)
- **Impervious surface raster**
 - CONUS Extent
- *TWI Raster (asking Mike W.)*
- **Flow Direction Raster (FDR)**
 - VPU-extent (Region 17)

Test Areas

- 17120004 (Southeast Oregon west of Malheur NWR)
- 10130103 (Eastern Missouri River basin, Prairie Pothole Country)

Things to do now (1/28)

- ~~• Write intermediate output to a directory, not geodatabase or geopackage~~
- ~~• As we start to proceed and get a good feel for QAQC, we can limit the number of things we write out~~
- ~~• Concentrate on the Tier 1 (pre-processing and data input) functions~~

- **We still need the TWI layer from Mike**
- ~~Segbuffer: Remove hard-coded buffer distances (30, 60) from segment buffer function, add as input variable~~
- ~~Hruimperv: Remove hard-coded impervious percentage for determining if a cell is impervious or not (50% or above); code as input variable~~
- ~~In getwaterbodiesinHRUsGrid – move the the reclassification of imprinting the HRU ID into waterbodies (lines 919 – 922) into the getwaterbodiesinHRUsGrid function, that way we can use different waterbody inputs (NHDPlusV2, HiRes, etc.~~
- Additional
 - **Second raster of waterbody IDs**

Ideas (don't implement for now)

- ~~Lower size limit for depressions (aka 120m²) (implemented)~~
- Differentiate NHD streams as on-stream storage in HRU, rather than just segments
- **For nhdpv2 waterbodies, see about getting upstream catchments from nhdpplustools rather than delineating (Andy)**
- Vary stream buffer by stream order

Variables

- ~~Raster cell size > use nhdpplusv2 rasters as the footprint and cellsize~~
- Paramdb > where to write csv parameter files (likely doesn't matter)

Intermediate Outputs

- ~~nhrug: raster version of HRUs~~
- ~~wbodg: raster version of waterbodies~~

Workflow

The workflow is broken up into 5 levels:

1. Level One: derivation of datasets that are used independently as inputs across all levels
2. Level Two: Determination of depression storage features and their upstream surface contributing areas
3. Level Three: Determination of surface runoff fractions to impervious, non-impervious, and on-stream areas
4. Level Four: Derivation of different contributing area maps and smidx and carea_max parameters that control runoff partitioning

5. Level Five : Profit

Level One – Preprocessing and setting environments (Finished 2/17)

1.—Set raster properties to nhrug for cell footprint and resolution

2.—Convert nhrug, nsegments, and waterbodies to rasters (nhrug, wbg)

3.—**getHruiImperv**

○—Generates Impervious raster labeled with HRU IDs

○—**Inputs**

▪—**nhrug**

▪—Impervious Surface Raster

▪—Impervious surface percentage (X)

○—For each cell with imperviousness over $> X\%$, assign a 1, else NULL

○—Populate Each cell with 1 with the hru_id

○—**Outputs**

▪—**hruImperv** : raster with impervious areas labeled with HRU IDs

4.—**getSegBuff**

a.—Buffers the stream segments, converts to grid, assigns HRU IDs within the buffer part of grid, NAs outside

b.—**Inputs**

i.—**Nhrug**

ii.—Nsegments vector

iii.—Buffer Distance

c.—Generate buffer around each nsegment, rasterize

d.—Populate Each cell within buffer with hru_id, outside buffer as NA

e.—**Output**

i.—**outEucDistance**

5.—**getwaterbodiesInHRUsGrid**

a.—Generates a HRU waterbody grid with each waterbody labeled with HRUIDs, then relabels with RegionGroup Function (not exactly sure what the output here is, may have to experiment with ArcPro functions “RegionGroup” and “Link”, see lines 611 area

b.—**Inputs**

i.—**Wbodg**

c.—Regions waterbodies for a minimum of 8, essentially the 8 cells surrounding the middle pixel in a 3x3 window; other arguments are kept as default. A link field is added which adds a unique identifier to each “grouped” waterbody

d.—This raster is derived with the “lookup” function applied to the “link” field

e.—**Outputs**

i. ~~wbodsHruGrid~~

Level Two – Delineating depression storage features and their upslope surface contributing areas

6. getDprst (Cory Start 2/17, Finished 3/31)

- a. Generates depression storage raster with depressions labeled as HRU IDs and all other areas with NULL. Depressions are only areas outside of impervious zones and outside the stream buffer.
- b. **Inputs**
 - i. **hruImperv**
 - ii. **outEucDistance**
 - iii. **wbodsHruGrid**
- c. **This is really the most confusing part of the code (lines 564 – 586)**
- d. Populate values from **wbodsHruGrid** for areas outside non-impervious area only (**getHruImperv**) output, output is called “junk1” in the code.
- e. Exclude junk1 from **getSegBuff** from junk1, this results in junk2 temporary layer (waterbodies outside impervious surfaces and streambuffer populated with their HRU IDs).
- f. Put waterbody link IDs from junk2 into a list
- g. Populate waterbodies outside the impervious areas and stream buffers with their HRU IDs (CON statement on 582)
- h. **Output**
 - i. **dprst**

7. getHruSro_to_dprst (Andy start 2/17, finished 3/31)

- a. Generates a raster of areas upstream of waterbodies/surface depressions
- b. **Input**
 - i. **Nhrug**
 - ii. **dprst**
 - iii. **FDR**
- c. Runs watersheds upstream of depressions returned from ‘getDpst’ function (“res1” in script, lines 191 to 197)
- d. Labels areas outside of depressions in dprst with 0 (“res2”), lines 200 – 209
- e. Uses a con (line 214) to put HRU IDs for watershed areas upstream of depressions (hruSro_toDprst grid name)
- f. **Output**
 - i. **hruSro_to_dprst**
- g. **Andy B.**

- i. Identify depressions based on featureid negative values for catchments
- ii. Identify waterbodies off of segments that are incorporated into the network topology, delineate upstream

8. ~~getDrpst_frac~~

a. ~~Possibly Deprecated/not used~~

Level Three: Determination of surface runoff fractions to impervious, non-impervious, and on-stream areas

Note: FDR raster is an input at this level

9. GetPervAreaTotal (Cory Starting week of 3/30)

a. Get raster with HRU IDs in areas that are not impervious or depressions.

b. Inputs

i. hrulmperv

ii. dprst

c. Basically a con operation to label stuff

d. Outputs

i. pervAreaTotal

10. get onStreamStor (Cory starting week of 3/30)

a. No quite sure of the point of this one.

b. Inputs

i. hrulmperv

ii. wbodsHruGrid

iii. dprst

c. Just two con statements, First identifies waterbodies outside of depressions, second outside of impervious areas, and returns those I guess

d. Output

i. nondprstWbodies

Level Four: Derivation of different contributing area maps and smidx and carea_max parameters that control runoff partitioning

Note : TWI raster is an input at this level

11. getCarea_map

a. Returns a raster of HRU IDs where there is pervAreaTotal and TWI is above a certain value, otherwise it returns null. For areas less than the TWI threshold that are also classified as onstream storage the HRU ID is also returned.

b. Inputs

- i. pervAreaTotal
- ii. TWI
- iii. OnStreamStor
- iv. Threshold (TWI threshold value)

c. Con Statement, where TWI exceeds thresholds, give HRU ID, else where there is onstreamstorage, give HRUID

d. Output

- i. Carea_map

12.getSro_to_dprst_perv

a. Surface runoff to depression storage for pervious surfaces. Returns proportion of pervious area draining to depression storage.

b. Inputs

- i. hruSro_to_dprst
- ii. pervAreaTotal

c. Creates raster representing pervious areas contributing to depressions

d. Rest of the code are table joins and counts. A lot of joins and such. We need to document this section better.

e. Outputs

- i. Sro_to_dprst_perv (dataframe)

13.getSro_to_dprst_imperv

a. Calculates Surface runoff to depression storage for impervious surfaces. Returns proportion of impervious area draining to depression storage.

b. Inputs

- i. hruImperv
- ii. hruSro_to_dprst

c. Basically same as 'getSRO_to_dprst_perv' but impervious

d. Outputs

- i. Sro_to_dprst_imperv (dataframe)

Level Five: Generating the parameters

14.getZonecount

a. Get Zone Counts from a grid per zone ID

b. Inputs

- i. Grid

c. Outputs

- i. **zoneCounts** (data frame)

15. **getCarea**

- a. Function : Computes proportion of pervious area that contributes to the stream
- b. **Inputs**
 - i. **threshold**
 - ii. **Carea_map (from threshold)**
 - iii. **PervAreaTotal**
- c. Derives carea_map with a given threshold (see **get_smidx** and **carea_max** function)
- d. Gets zone count for **perviousAreatotal** by HRU
- e. Get zone count for derived **carea_map** for given threshold
- f. Does some joins and easy math
- g. **Outputs**
 - i. **Carea**(dataframe)

16. **getSmidx**

- a. Computes Soil Moisture Index Coefficient with getCarea function and a TWI threshold of 15.6
- b. Returns dataframe

17. **getCarea**

- a. Computes Soil Moisture Index Coefficient with getCarea function and a TWI threshold of 8
- b. Returns dataframe

18. **getHRU_percent_imperv**

- a. Compute the impervious fraction of the HRUs.
- b. **Inputs**
 - i. **hrulmperv**
- c. Weird stuff, maybe we can do it better
- d. **Output**
 - i. **dataframe**